

4D SCENE GENERATION → WORKFLOW :

1. Cloud GPUs with control

- AWS EC2 (p4d/p5 instances), GCP (A100/H100 VMs), Lambda Labs, Paperspace, CoreWeave.
- Gives you **dedicated GPUs** that don't disconnect.

2. Containerization

- Package 4DNeX in **Docker** with CUDA base image.
- Makes deployment repeatable across servers.

3. Inference server

- Use **FastAPI** or **Flask** to expose 4DNeX as a REST API.
- Add a **queue system** (e.g., Celery, Redis) if many requests come in.

4. Storage & delivery

- Dynamic point clouds/videos can be big → need **S3/GCS buckets** or similar to store and deliver results.

5. Monitoring & scaling

- Kubernetes (K8s) or serverless GPU platforms let you spin up more GPUs as demand grows.

❖ How I am working on it :

1. **Prototype on Colab** → show that you can generate nice dynamic point clouds/videos.
2. **Migrate to a dedicated GPU server** (Lambda Labs, CoreWeave, AWS).
3. **Containerize (Docker)** the code + models.
4. **Wrap in an API** → so your web/app can send image → get video/point cloud back.
5. **Add monitoring + scaling** before launching publicly, using (Kubernetes (K8s) or serverless GPU).